

Day - 42

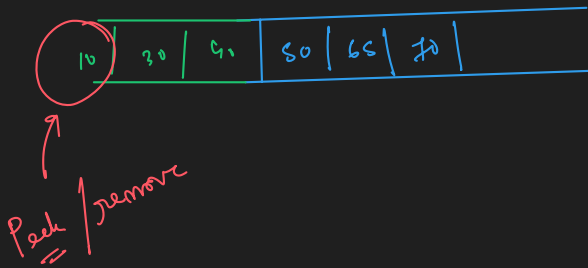
Heap or Priority Queue

max
min (default)

min PB

50 65 70 10 30 40

data will insert or remove based on priority.



add() $\rightarrow O(\log n)$

remove() $\rightarrow O(\log n)$

peek() $\rightarrow O(1)$

using JCF

PriorityQueue<Integer> PQ = new PriorityQueue<>(); // asc. order.

PriorityQueue<Integer> PQ = new PriorityQueue<>(Comparator.reverseOrder()); // desc order.

this $\rightarrow$ s1

this $\rightarrow$
Student s1 = ('Prayan', 10)
Student s2 = ('Dhruv', 15) ✓

```
int compareTo (Student s2) {
    return this.rank - s2.rank;
}
```

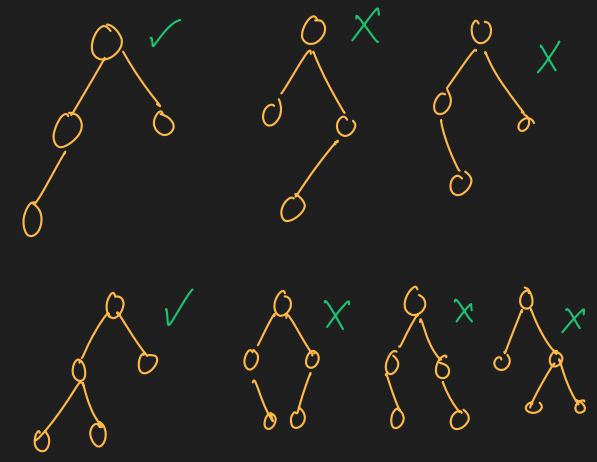
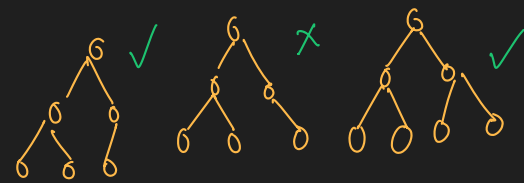
s1 s2 $\rightarrow$ -ve $s_1 < s_2$
+ve $s_1 > s_2$
= $s_1 = s_2$

Heap $\rightarrow$ max [max ele $\uparrow$, priority $\uparrow$]
min (default) [min ele $\downarrow$, priority $\uparrow$]

visualised $\rightarrow$ BT (complete BT).
implemented as array or AL.

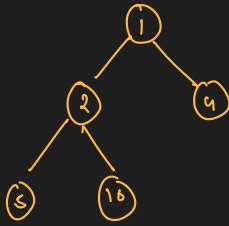
Property

① Complete BT
[cBT is a BT which levels are completely filled except possibly the last one, which is filled from left to right]



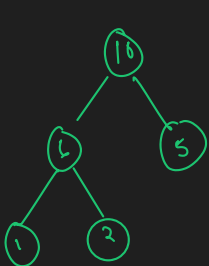
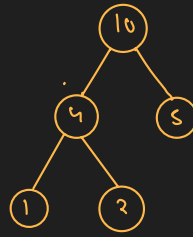
② for min heap

children > Parent

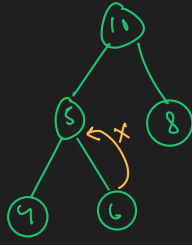


for max heap

children < Parent

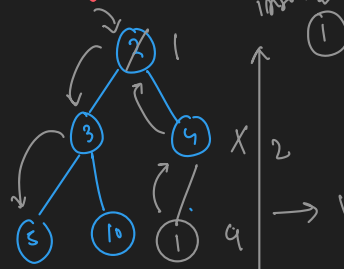


[max heap]



[not a heap]

why heap is not implemented as tree



add $\rightarrow O(n)$

adjustment $\rightarrow O(\log n)$

Parent $\rightarrow$ Child relⁿ

parent(i)
child = $2i+1$
 $= 2i+2$

$p \approx i = 0$
 $c_1 = 1$
 $c_2 = 2$

$p = i = 1$
 $c_1 = 3$
 $c_2 = 4$

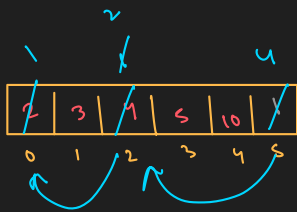
Child $\rightarrow$ Parent relⁿ

child = n

parent = $(n-1)/2$

to make a heap $\rightarrow O(n \log n)$

Implement as Array or AL



insert ①

$5-1/2 = 2$

$2-1/2 = 0$

insert(ele) $\rightarrow O(\log n)$

Step 1

add at last position in AL/Array

Step 2

adjust the heap.

while (child val < parent val) {
 swap(child, parent);
}

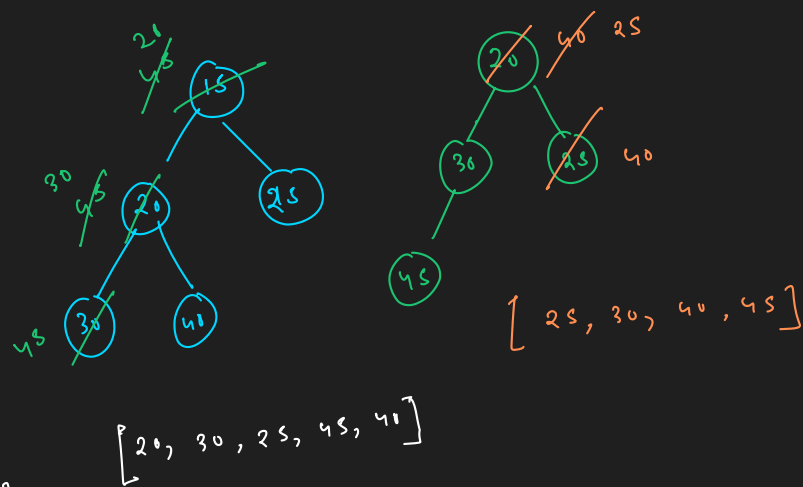
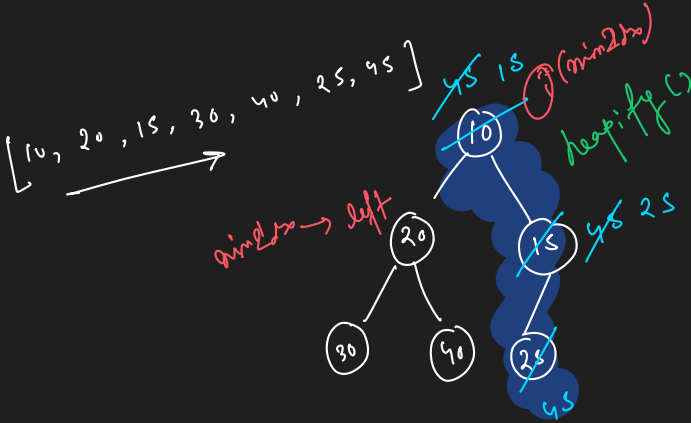
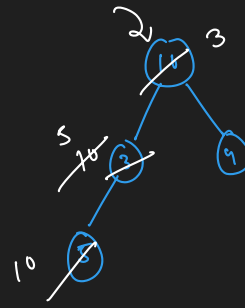
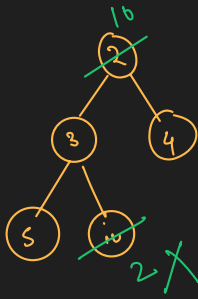
```
// add()
public void add(int data){
    arr.add(data);

    // adjust the heap
    int c = arr.size()-1;
    int p = (c-1)/2;

    while (arr.get(c) < arr.get(p)) {
        // swap
        int temp = arr.get(c);
        arr.set(c, arr.get(p));
        arr.set(p, temp);

        // update
        c = p;
        p = (c-1)/2;
    }
}
```

Remove



[15, 20, 25, 30, 40, 45]

[20, 30, 25, 45, 40]

Step 1 swap first and last position

Step 2 we remove the value of the last index

Step 3 fix the heap using `heapify()`.

```
// remove()
public int remove(){
    // step 1
    int temp = arr.get(index: 0);
    arr.set(index: 0, arr.get(arr.size()-1));
    arr.set(arr.size()-1, temp);

    // step 2
    int data = arr.remove(arr.size()-1);

    // step 3
    heapify(i: 0);

    return data;
}
```

```
public void heapify(int i){
    int left = 2*i+1;
    int right = 2*i+2;

    int minIdx = i;

    if(left < arr.size() && arr.get(left) < arr.get(minIdx)){
        minIdx = left;
    }
    if(right < arr.size() && arr.get(right) < arr.get(minIdx)){
        minIdx = right;
    }

    // swap
    if(minIdx != i){
        int temp = arr.get(i);
        arr.set(i, arr.get(minIdx));
        arr.set(minIdx, temp);

        heapify(minIdx);
    }
}
```

H.W Max Heap